COS214: TIVIDY

Ethan Van Eyden (u25260244)
Christian Nunes (u25138309)
Stefan van Staden (u25031784)
Lesego Tebeile (u25143230)
Jayden Dean (u24647897)
Kyle Pillay (u23545756)
Rochaan Verster (u25045785)

Table of Contents

Task 1: Research 2
Task 2: Scenario definition and Scenario 3
Task 3: UML activity diagrams 4
Task 4: Design Patterns 5
Participant mapping 5
Task 5: UML Class diagram 10
Task 6: Runtime Behaviour Diagrams 11
Task 7: Design Decisions and Revisions 12

Task 1: Research

A workflow is a computerized representation of a business process. It lists mandatory activities, their order, conditions, necessary data, users, and needed tools. Now a Workflow Management System is a system that is used to create, execute and monitor workflows. It coordinates the people and activities involved in a business process.

A workflow's definition differs from its instance as a definition describes how the process should work whereas an instance is one actual execution of the process, allowing for many instances of the same process to run at the same time and at different stages.

A workflow is made up of individual activities. Each activity can specify what needs to be done, who performs it, what tools are needed, and how it connects to other activities.

Work is assigned to users based on roles and staff assignments. The workflow system determines where an activity should be performed and can route work between different users.

Workflows need to support different paths depending on decisions or outcomes. If an activity cannot continue normally, the workflow should be able to follow an alternative path or compensate for previous work.

Workflows can involve both **people and computer programs**. The system needs information about roles, staff assignments, and organizational structures so that work can be given to the correct participants.

A WFMS must handle failures so that information is not lost and the entire process does not stop. It may need to reroute work to another available component. A WFMS needs to coordinate and integrate different applications, databases, and other technologies within the same workflow. This is especially important in large organizations with different systems and platforms.

The workflow definition should be separate from its individual instances. This allows the same definition to be reused for many different executions. This makes the system suitable for different business processes rather than only one specific process.

Workflow processes cannot predict every possible exception. Therefore, the system needs flexible navigation so that different paths can be selected when unexpected situations occur rather than having every situation permanently hard-coded.

References

1. Gertrude Kappel, Peter Lang, Stefan Rausch-Schott & Werner Retschitzegger (1995), Workflow Management Based on Objects, Rules, and Roles, IEEE Bulletin of the Technical Committee on Data Engineering, Vol. 18(1), pp. 10–17.

http://sites.computer.org/debull/95MAR-CD.pdf#page=20

2. Georgakopoulos, D., Hornick, M. & Sheth, A. (1995). An Overview of Workflow Management: From Process Modeling to Workflow Automation Infrastructure. Distributed and Parallel Databases, 3, 119–153. DOI: 10.1007/BF01277643.

https://link.springer.com/content/pdf/10.1007/bf01277643.pdf

3. Schulz, K.A. & Orlowska, M.E. (2004). Facilitating cross-organisational workflows with a workflow view approach. Data & Knowledge Engineering, 51, 109–147.

https://doi.org/10.1016/j.datak.2004.03.008

4. Kumar, A., van der Aalst, W.M.P. & Verbeek, E.M.W. (2002). Dynamic work distribution in workflow management systems: How to balance quality and performance. Journal of Management Information Systems, 18(3), 157–193. DOI: 10.1080/07421222.2002.11045693.

https://www.tandfonline.com/doi/pdf/10.1080/07421222.2002.11045693

Task 2: Scenario definition and Scenario

Initial scope of TIVIDY:

TIVIDY is a generic workflow management system. It stores definitions, creates independent running instances from them, and coordinates the work, participants, rules and events needed to carry each instance outcome.

Main responsibilities:

Building and validating definitions; creating and starting instances; representing work as nested groups and individual items; assigning work using configurable rules; managing the lifecycle of work and the phases; routing results through approval or rejecting and/or escalations; react to internal and external events and handle according to intent; record history and have a rollback method.

Boundaries:

TIVIDY decides who does the work, in what order and what happens with the result of said work. It does not perform the work itself, it does not own external systems- only talks to them through external connectors, and it does not hard code any one organization's rules.

2.2 Scenario: Assignment Marking in a University Module

Environment: a university Computer Science department running a large module with hundreds of students, marked by several assistant lecturers (ALs).

Process: the full life of one assignment: defining it, collecting submissions, marking in parallel, moderating, approving and releasing marks.

Participants: module coordinator (defines the assignment, final review, approves release); lecturer (reviews after a first rejection); assistant lecturers (mark and re-mark); students (submit and receive marks).

Major work: define the assignment, collect submissions, mark each submission question by question, review marks, re-mark rejected scripts, approve, release marks. Structure: module → assignment → student submission → questions.

Decisions and routing: Was the submission on time? Which AL is assigned? Are marks accepted or rejected? Who reviews next (lecturer after 1 rejection, coordinator after 3)?

Was the marking deadline missed (escalate)? Is everything accepted so the phase can advance?

Approvals and dependencies: marking starts only after collection closes; review only after marking; release only after all scripts are accepted and the coordinator approves.

Failures and exceptions: late or missing submissions; an AL missing the deadline or being unavailable; repeated rejection; the LMS being unreachable at release (retry, then fail); invalid phase changes (e.g. releasing before approval) being refused.

External systems: an LMS (submission intake, mark release) and an email service (notifications).

Expected outcome: all submissions are marked, reviewed and approved, marks are released to students, and a full audit history shows who did what and when. Items that cannot be completed end as rejected or failed and are flagged for follow-up.

Education Scenario (Assignment Marking)

· The Setup: Students submit assignments (Items) to an Assignment (Container). TAs (Workers) are auto-assigned to mark them.

· The Flow: TAs enter a mark (0-100). Once all are marked, the Assignment enters the Review phase.

· The Chain: The Lecturer (Operator) clicks "Approve Batch." The chain auto-approves marks between 40 and 100. The person who approved it depends on the mark. TAs can approve 50-70, lecture can do above 70 and moderator can do fringe like 40-50.

· The Rejection: If a TA gives a mark below 40, it becomes Rejected. The TA does not re-mark it. The Lecturer must personally read the paper and either accept the TA's low mark or enter their own.

· The End: Once all submissions are approved, the Assignment publishes and marks are released to students.

Healthcare Scenario (Patient Treatment)

· The Setup: Patients arrive for Treatment (Items) in a Ward (Container). Doctors (Workers) are auto-assigned to treat them.

· The Flow: Doctors examine patients, record a diagnosis, and enter a severity level (1-10). Once all treatments are done, the Ward enters the Review phase.

· The Chain: The Chief of Medicine (Operator) clicks "Approve Batch." The chain auto-approves valid severities (1-10).

· The Rejection: If a doctor enters an invalid severity (e.g., 0), it becomes Rejected. The doctor does not re-examine the patient. The Chief must personally examine the patient and either accept the doctor's record or enter their own severity.

· The End: Once all treatments are approved, the Ward publishes. Beds are assigned, and patients are officially admitted/discharged.

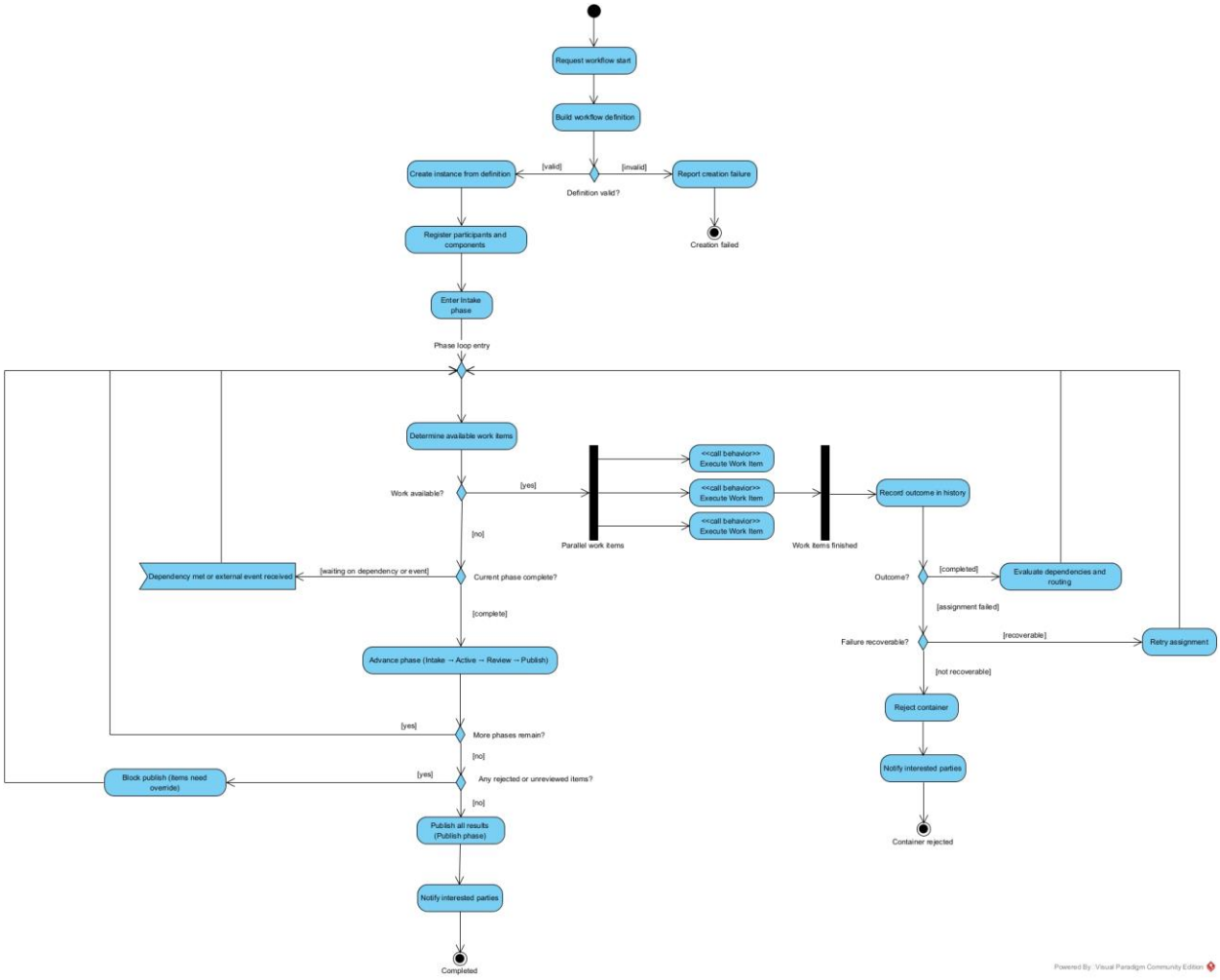
Shared Rules (Both Scenarios)

· No Rework Loop: Rejection is not a "send back to worker" button; it is an "escalate to the boss" button. The boss does the work.

· Atomic Publish: The Container (Assignment/Ward) cannot publish while any item is Rejected. The Operator must resolve every flagged item first.

· Review Tracking: The item records both who did the work (assignee) and who approved it (reviewer), both as Participant*.

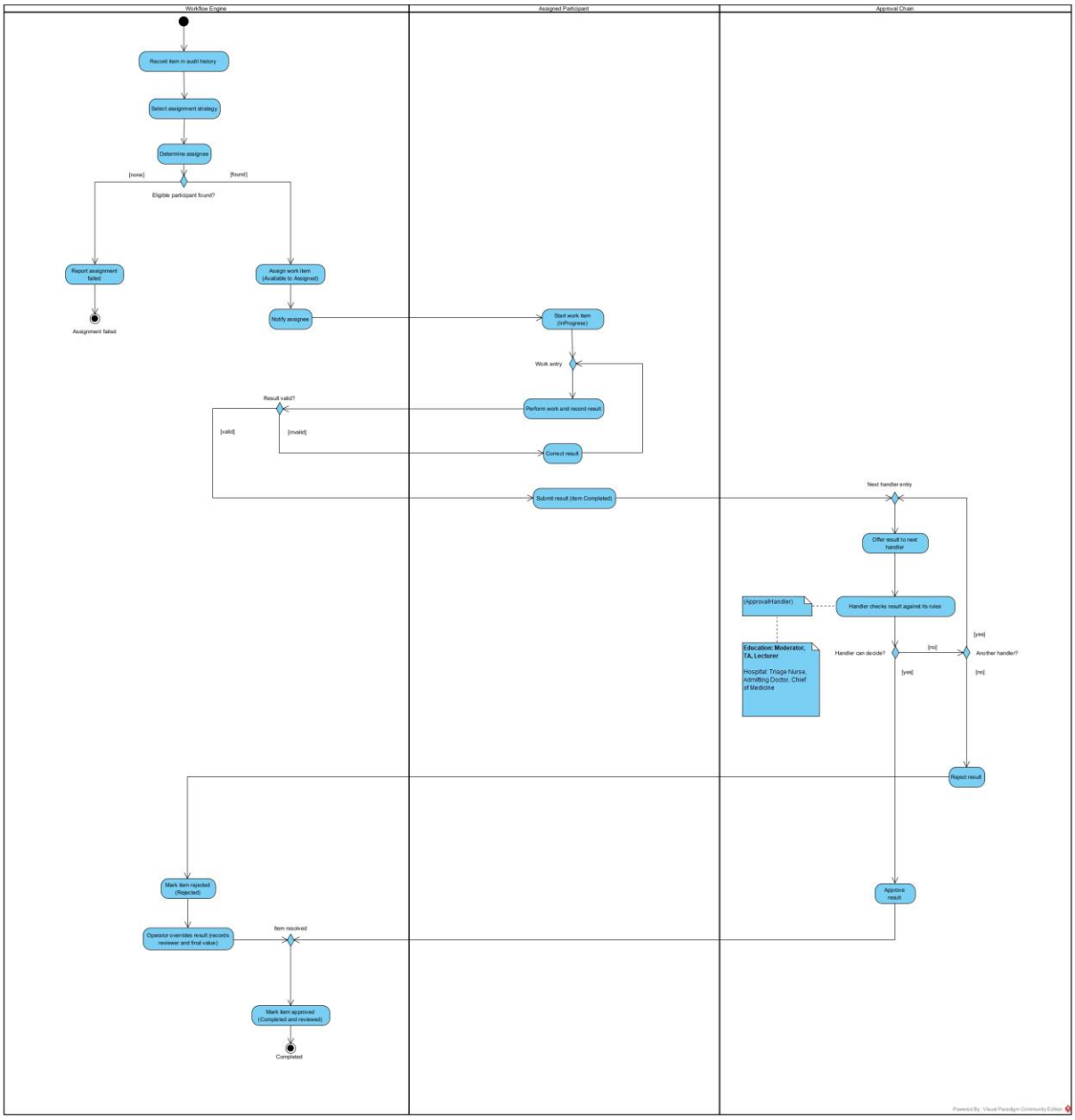
Task 3: UML activity diagrams



Run Workflow Instance

Shows the life of one running workflow instance. A workflow definition is built and validated, and an instance is created from it. Participants are registered and the instance enters its first phase, Intake. In each phase the engine finds the available work items and runs them in parallel through the called activity Execute Work Item. It records each outcome in the history and evaluates dependencies. If an assignment fails, it is retried, or the container is rejected and interested parties are notified. External events can resume a phase that is waiting on a dependency. When a phase is complete the container advances through Intake, Active, Review and Publish. Before publishing, the engine checks that no item is rejected or unreviewed. If one is, publishing is blocked until it is resolved, and the

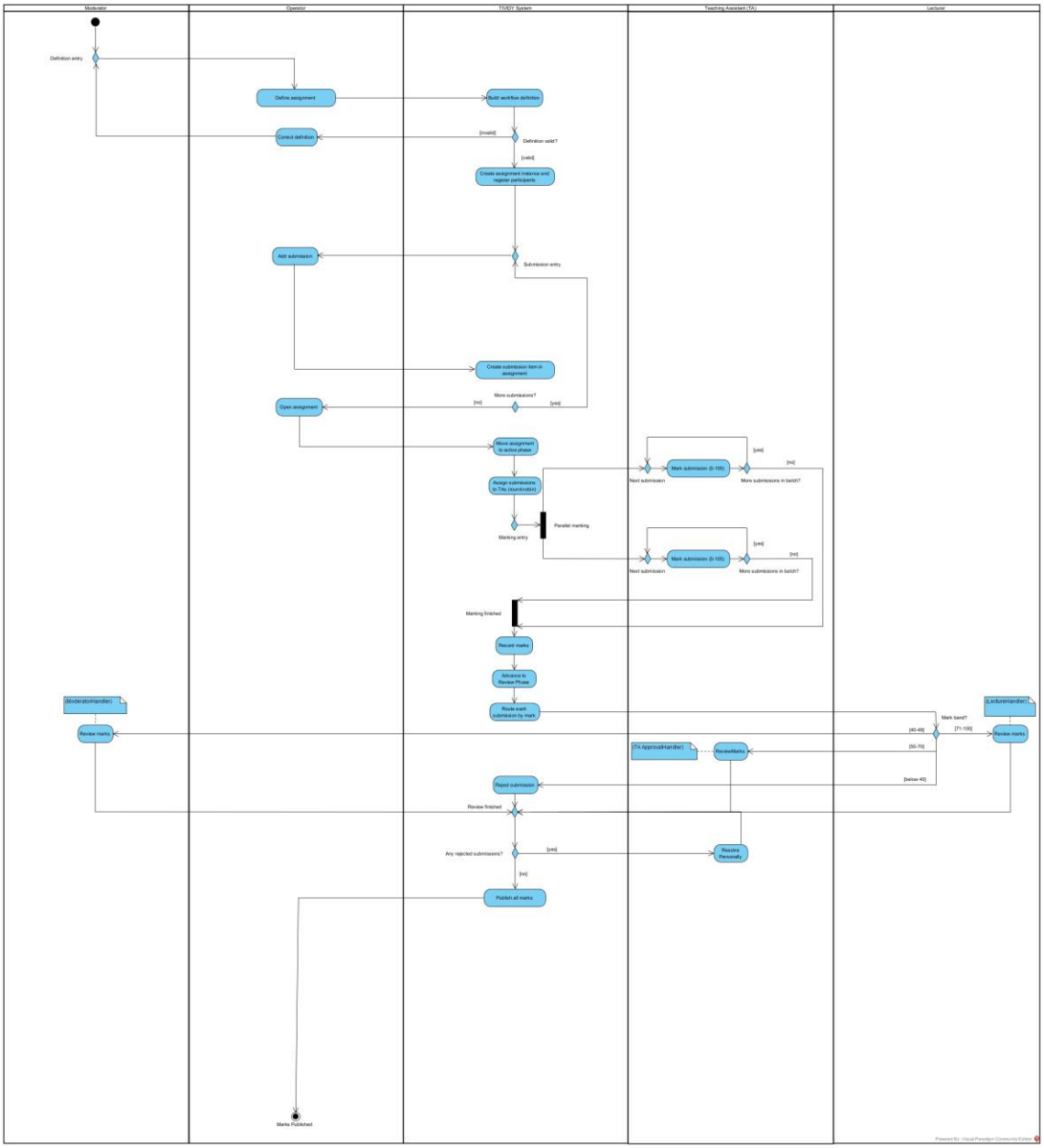
instance completes only once everything is published together.



Execute Work Item

Details one work item from assignment to its final outcome, across three swimlanes. The engine records the item in the audit history, selects an assignment strategy and chooses an assignee. If no eligible participant exists, assignment fails. Otherwise the item moves from Available to Assigned and the assignee is notified. The participant starts the work (InProgress), performs it, corrects an invalid result, and submits it, which moves the item to Completed. The approval chain then offers the result to each handler in turn. A handler

approves it if the result falls within its rules, and otherwise passes it to the next handler. If no handler can decide, the result is rejected and the item is marked Rejected. An operator then overrides it and records the final value, so the item ends as approved. Handlers are scenario-specific: Education uses Moderator, TA and Lecturer, and Hospital uses Triage Nurse, Admitting Doctor and Chief of Medicine.



Marking Workflow (Education)

Applies TIVIDY to assignment marking, from creating the assignment to publishing the marks. The operator defines the assignment, the system builds the workflow definition, and the operator adds the student submissions. The assignment then moves from Intake to Active. Submissions are assigned to teaching assistants in turn, and the TAs mark them in parallel (0 to 100). When all marking is finished the assignment advances to Review, and each mark is routed by its band. Marks of 40 to 49 are approved by the Moderator, 50 to 70 by a different TA, and 71 to 100 by the Lecturer. A mark below 40 has no approver and is rejected. If any submission is rejected, the Lecturer resolves it personally by accepting the TA's mark or entering their own. Once no rejected submissions remain, all marks are published together.

Task 4: Design Patterns

Participant mapping

Patterns:

Composite:

Component: Component
Composite: Container

Leaf: item

State: (For Containers)

Context: Container

State: WorkFlowPhase

Concrete State:

IntakePhase, ActivePhase, ReviewPhase, PublishPhase

State: (For Items)

Context: Item

State: ItemState

Concrete State:

CreatedState, AssignedState, InProgressState, CompletedState, RejectedState

Strategy:

Context: AssignmentEngine

Abstract Strategy: AssignmentStrategy

Concrete Strategy: RandomStrategy, WorkloadStrategy, RoundRobinStrategy

Command: WorkCommand

ConcreteCommand: AssignItemCommand, RecordResultCommand, RejectItemCommand, OverrideItemCommand, AdvancePhaseCommand, ApproveContainerCommand, SystemLogCommand

Invoker: ActionHistory

Receiver: Item, Container

Iterator:

Iterator: Iterator

ConcreteIterator: DepthFirstIterator, StateFilterIterator

Aggregate: Component

ConcreteAggregate: Item

Mediator:

Mediator: WorkflowMediator

Colleague: Colleague

ConcreteMediator: WorkflowCoordinator

ConcreteColleagues:

WorkContainer (Asks to advance states)

WorkItem (Reports when result is recorded)

AssignmentEngine (Mediator tell when to assign strategy to items)

ActionHistory (Records actions when told to by mediator)

Observer:

Subject:EventBus

ConcreteSubject: EventBus

Observer: Observer

ConcreteObserver: EducationObserver, HealthcareObserver

Builder: Create workflow()

Builder: WorkflowBuilder

Director: AssignmentMarkingDirector, HospitalAdmissionDirector

ConcreteBuilder: AssignmentMarkingBuidler, HospitalAdmissionBuilder

Product: Root

Chain of Responsibility:

Handler: ApprovalHandler

Concrete Handlers (education) = ModeratorApprovalHandler , TAApprovalHandler, LecturerApprovalHandler;

Concrete Handlers (hospital) = TriageNurseHandler, AdmittingDoctorHandler, ChiefOfMedicineHandler.

EducationBandHandler and HospitalSeverityHandler are shared intermediate bases. EducationApprovalChain and HospitalApprovalChain assemble and link the handlers.

Abstract Factory:

Abstract Factory: WorkflowComponentFactory;

Concrete Factories:EducationWorkflowFactory, HospitalWorkflowFactory;

Abstract Products: WorkflowBuilder, AssignmentStrategy

Façade:

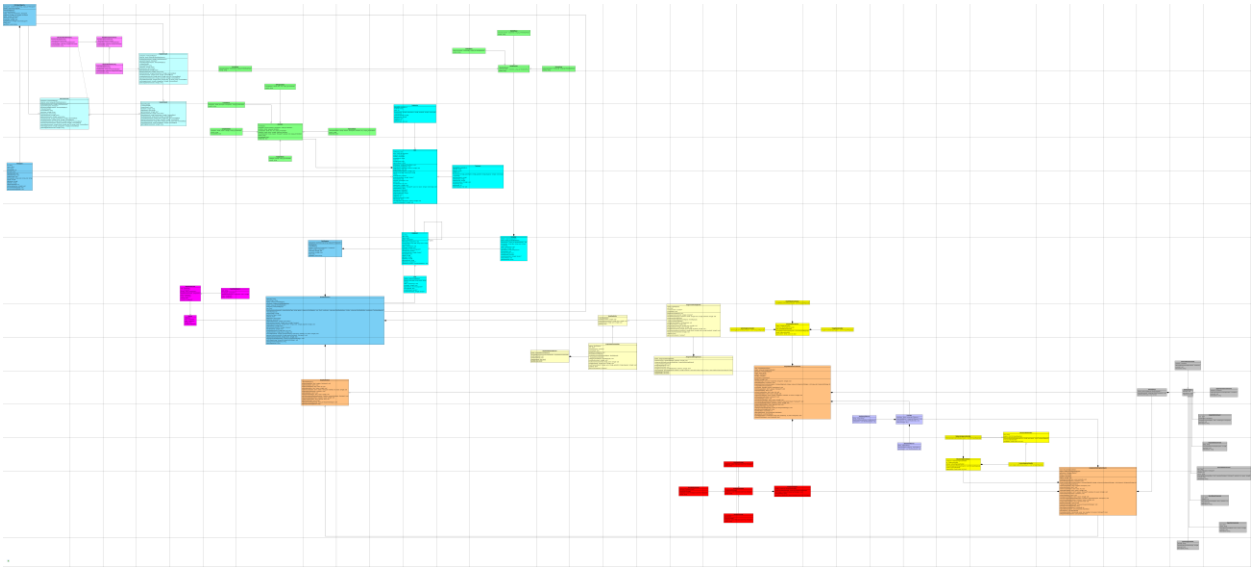
Facade: TividyWorkflow

Subclasses: WorkflowInstance, WorkflowDirector / WorkflowBuilder, WorkContainer and WorkItem, AssignmentEngine, ActionHistory, WorkflowCoordinator, ApprovalHandler, EventBus, Workregistry

Client: RestServer

Task 5: UML Class diagram

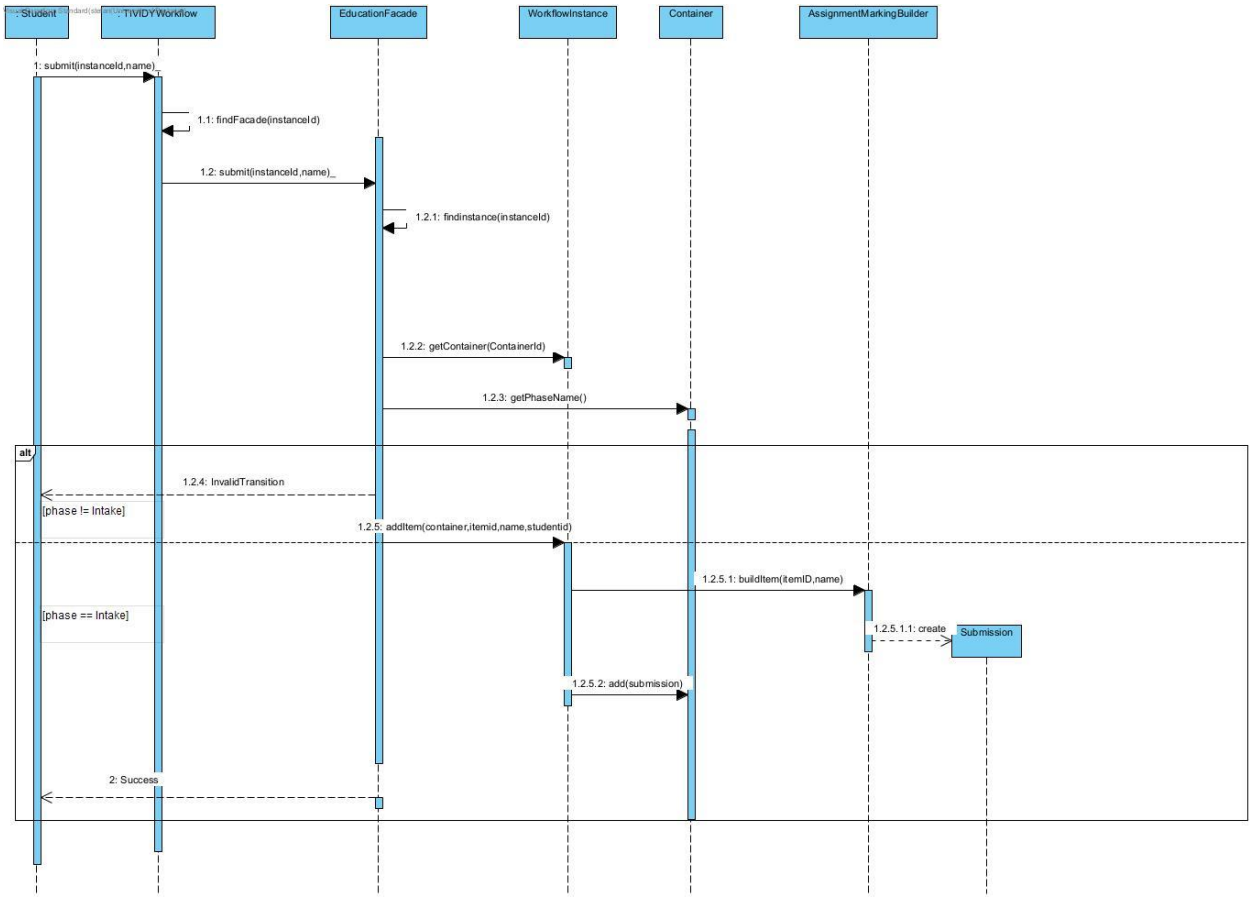
For a better experience please view the pdf/svg image in the github repo.



Task 6: Runtime Behaviour Diagrams

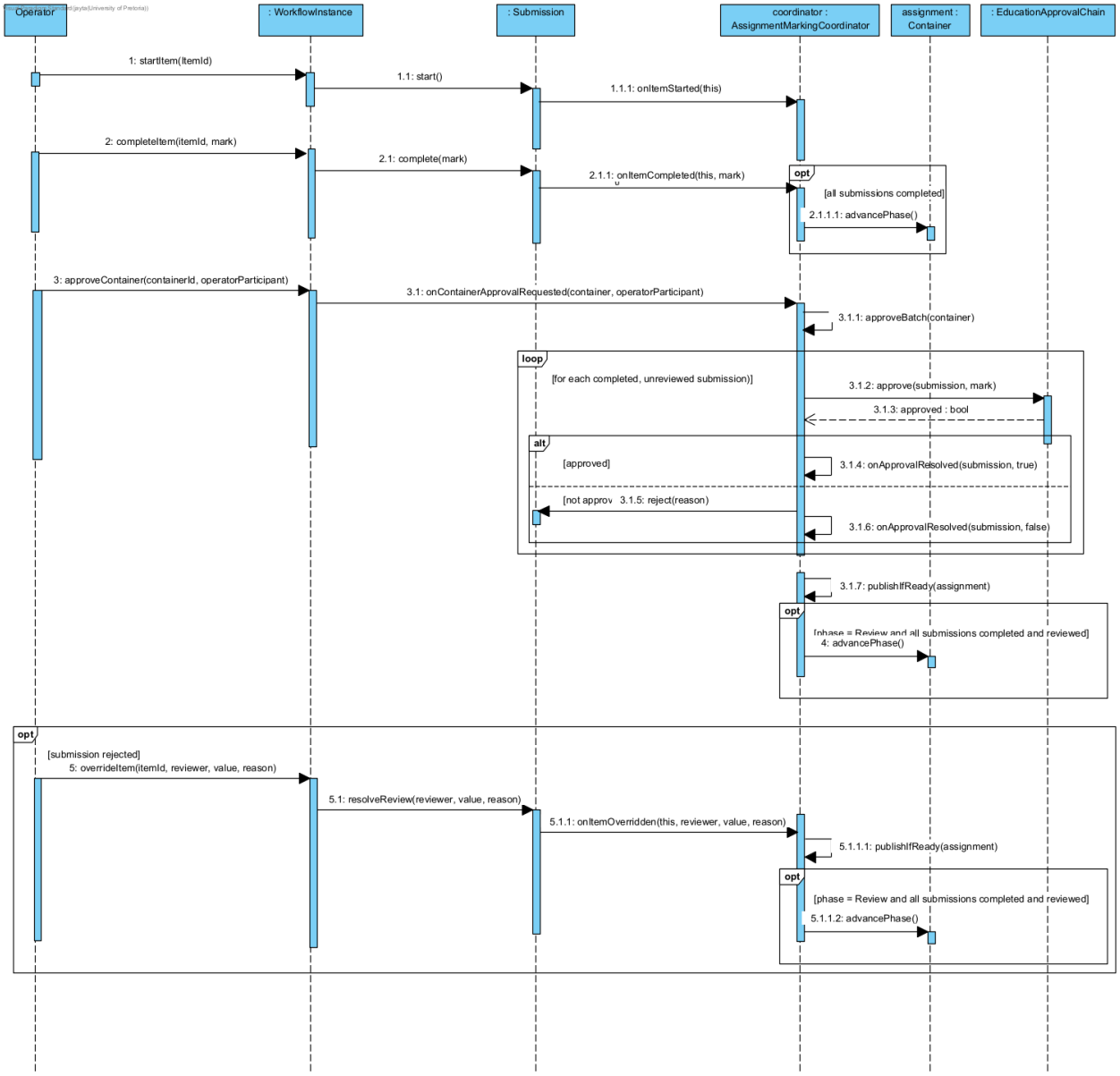
SD1: Assignment Submission

A student submits an assignment through TIVIDY. TIVIDY checks whether the assignment's submission window is still open (verifying the assignment container is currently in the "Intake" phase). If the intake phase has passed, the submission is rejected. If the submission is valid, TIVIDY creates a new Submission and adds it to the assignment's Container within the existing Workflow Instance to await marking.



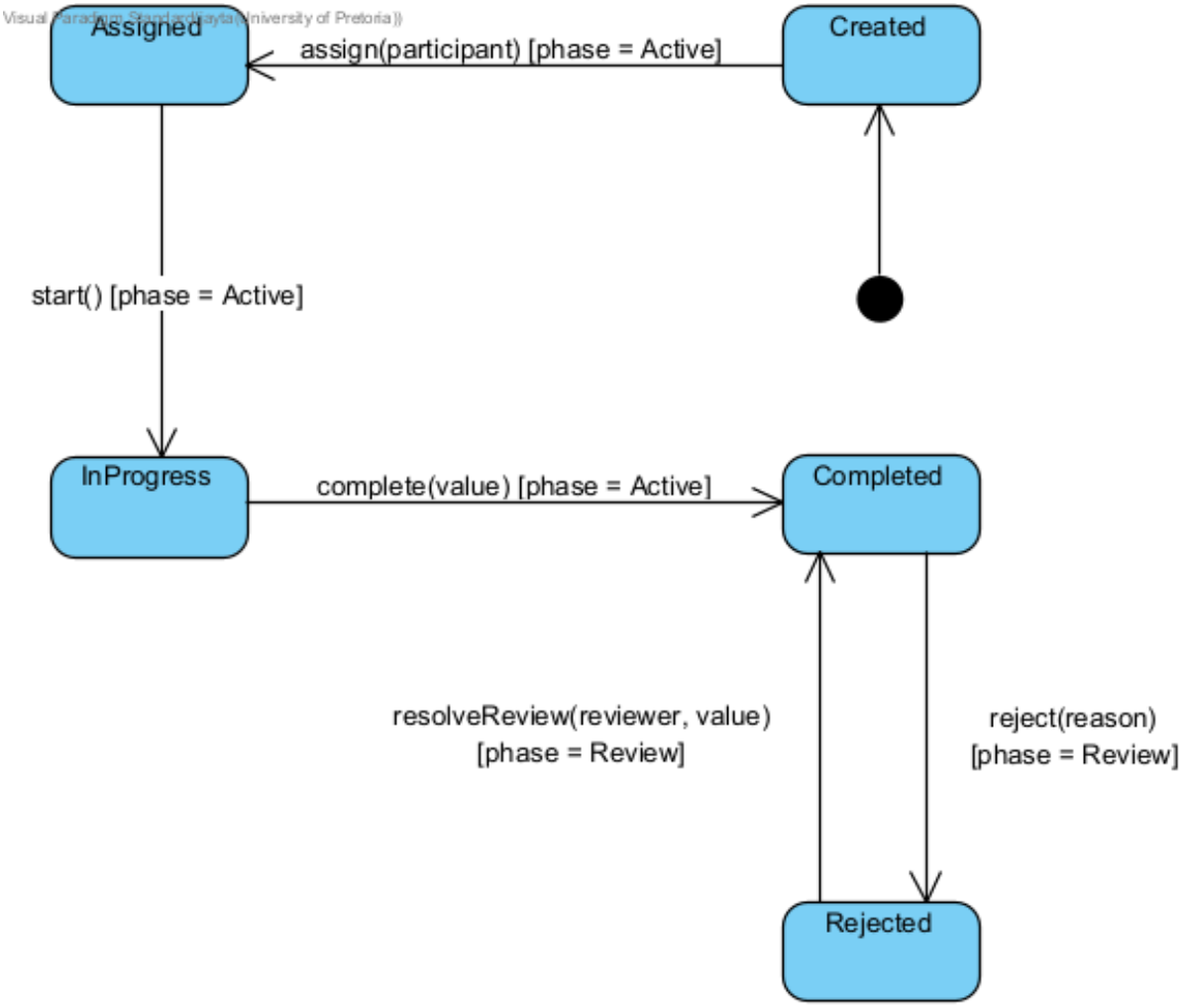
SD2: Marking, Review and Release

After submissions close, participants start marking their assigned submissions and record the results. Once all the submissions are marked, the assignment moves to Review. TIVIDY sends each completed, unreviewed submission through the approval chain. If a result is rejected, personal intervention is required to accept or correct it, returning the submission to Completed with a recorded review. Once all submissions are completed and reviewed, the assignment advances to Publish.



State Diagram: WorkItem State Diagram

We chose the Item as a stateful object because its behavior changes throughout the workflow. It starts in Created, becomes Assigned when a participant is selected, moves to InProgress when working begins. Completing the work records a result and changes the Item state to Completed. In the review phase, if something with the result is wrong and needs personal intervention, the Item may be Rejected. Once the necessary adjustments are done, the Item moves back to Completed.



State Diagram Interpretation

- **Created:** The marking Item has been created but has not yet been assigned.

- **Assigned:** A participant has been selected. Item is assigned to them.
- **In Progress:** The participant is actively working on the item.
- **Completed:** The participant has completed the appropriate task, and the result has been recorded; the result may require review.
- **Rejected:** The work needs review and may need adjustment. Personal intervention is needed to resolve it.

Github:

https://github.com/ethanvaneyden/TIVIDY

Build/run/test Instructions:

#Optionally if u want to start afresh

rm -rf build

1. Configure (once; re-run after changing CMakeLists.txt)

cmake -S . -B build

2. Build everything

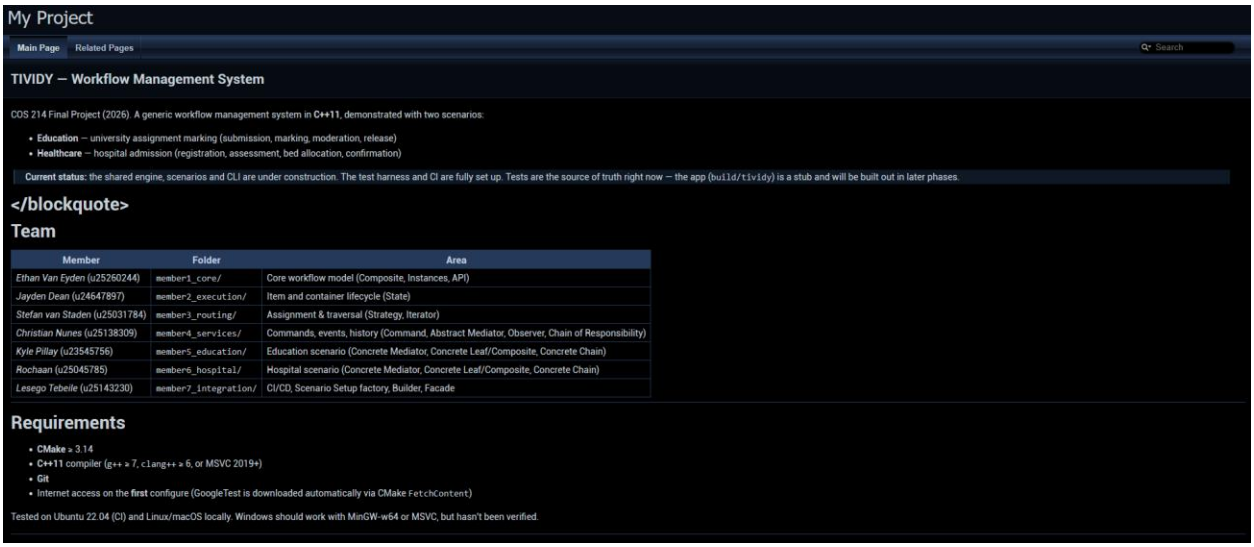
cmake --build build --target tividy_tests

3. Run the full test suite

ctest --test-dir build --output-on-failure

Run:

./build/tividy




```
/**
 * @brief Base class for objects that communicate through a Mediator.
 *
 * Inherited by Component, AssignmentEngine, ApprovalHandler, etc.
 * Holds a non-owning pointer to the Mediator.
 */
class Colleague {
protected:
    WorkflowMediator* mediator = nullptr;
```

Contribution:

Member 1 — Core workflow model

Implement:

- WorkComponent — common interface for individual and grouped work.
- WorkItem — an individual task, including its identifier, description, payload/result, assignee reference, deadline and current state reference.
- WorkContainer — holds nested work components and provides a common way to inspect child work.
- WorkflowDefinition — reusable definition containing workflow structure, dependencies, configuration and routing rules.
- WorkflowInstance — unique instance ID, reference to its definition, runtime work structure, current progress, instance data and status.
- Basic participant and role abstractions, agreed with Member 3.

Patterns: Composite; part of Iterator's aggregate structure.

Member 2 — Workflow instances, lifecycle and progression

Execution core

Implement:

- WorkflowEngine or the agreed execution service — creates and starts instances.
- WorkflowPhase and phase implementations where required.
- WorkItemState and concrete states such as CreatedState, AssignedState, InProgressState, CompletedState, RejectedState and CancelledState.
- Dependency evaluation — determine when downstream tasks become eligible.
- Generic routing/branch progression using configured conditions or rules.

Patterns: State.

Member 3 — Assignment, policies and work traversal

Routing services

Implement:

- Participant and Role, in agreement with Member 1.
- AssignmentEngine — uses a configured strategy to assign eligible work.
- AssignmentStrategy interface.
- RoleBasedStrategy, WorkloadStrategy and RoundRobinStrategy, as feasible for your scope.
- WorkIterator, DepthFirstIterator and, if needed, PriorityIterator.
- Generic policy interfaces for deadlines, eligibility or routing where they are genuinely useful.

Patterns: Strategy and Iterator.

Member 4 — Commands, coordination, events and history

Cross-cutting services

Implement:

- WorkCommand interface and CommandInvoker.
- Commands such as AssignItemCommand, CompleteWorkItemCommand, RecordResultCommand, ApproveResultCommand, RejectResultCommand and AdvancePhaseCommand, with suitable generic boundaries.
- WorkflowMediator and WorkflowCoordinator.
- EventBus, WorkflowEvent, WorkflowObserver and observer registration/removal.
- ActionHistory and audit records for actions, actors, timestamps, outcomes and relevant state changes.
- A generic ApprovalHandler interface and chain mechanism, if agreed with the scenario owners.

Patterns: Command, Mediator, Observer and Chain of Responsibility.

Member 5 — Education scenario implementation

Scenario A

Implement:

- AssignmentMarkingBuilder implementing the agreed workflow-builder interface.
- Assignment-specific input and result data, including submission metadata, marking results and review outcomes.
- Education-specific task definitions and dependencies.
- Marking assignment configuration, moderation rules and rejection-count routing.

- Education-specific approval handlers or policies.
- An LMS adapter or stub and, where appropriate, email notification integration.
- Scenario data for students, assistant lecturers, lecturer and module coordinator.

Patterns: Builder; Adapter for external integrations; Chain of Responsibility through the agreed approval mechanism.

Member 6 — Hospital admission scenario implementation

Scenario B

Implement:

- HospitalAdmissionBuilder implementing the same builder interface as Member 5.
- Hospital-specific admission data, such as admission request ID, required documents, assessment outcome, bed requirements and allocation status.
- Registration, validation, assessment, bed allocation and confirmation tasks.
- Hospital-specific eligibility and routing configuration.
- Referral/escalation handlers and rules for unavailable staff or beds.
- A hospital-system adapter or stub, and optional notification integration.

Patterns: Builder; Adapter; Chain of Responsibility through the agreed referral/escalation mechanism.

Member 7 — Integration, factories, decorators and runnable backend

Integration owner

Implement:

- WorkflowBuilder interface and WorkflowDirector, in coordination with Members 5 and 6.
- WorkflowComponentFactory and concrete scenario factories, but only if the factory creates meaningful families of compatible components.
- TividyWorkflow facade, providing a consistent API to start workflows, inspect work, execute commands and retrieve history.
- WorkDecorator plus PriorityDecorator, AuditDecorator and DeadlineDecorator, coordinating with Members 1 and 4.
- Integration entry point and a minimal CLI/demo driver to select either scenario and interact with the backend.
- Common build configuration, such as a Makefile or CMakeLists.txt, and integration fixes.

Patterns: Facade, Decorator and Abstract Factory, if justified by the actual implementation.